

Day 125: Basic API Testing and Authentication

1. Day number

Day 125

2. Topic name

API Testing, Authentication, and API Keys

Today you will learn how to check whether an API behaves correctly and how some APIs identify callers before allowing access.

3. Connection

Yesterday, on **Day 124**, you learned how to send requests manually using Postman:

```
Choose method
  ↓
Enter URL
  ↓
Add headers/body if needed
  ↓
Click Send
  ↓
Inspect response
```

Today, you will go one step further.

Instead of sending only a successful request, you will deliberately test different situations:

```
Valid request
  ↓
Should succeed

Invalid request
  ↓
Should fail correctly

Missing authentication
  ↓
Should be rejected when authentication is required
```

This is the beginning of **API testing**.

4. Important topics

Today you will focus on:

- successful request
- failed request
- API key
- authentication
- Authorization header concept
- environment variable idea

You will continue using the simple HTTP and Postman knowledge from the previous days.

5. Foundational notes

API testing means checking:

Does the API behave the way we expect in different situations?

Suppose you have:

```
GET /products/10
```

If product 10 exists, you might expect:

```
200 OK
```

with product data.

But good testing does not stop there.

You should also ask:

What happens if the product ID is missing?

What happens if the data is invalid?

What happens if authentication is required but missing?

What happens if the API key is wrong?

So API testing includes both:

Happy path

↓

Things work correctly

Failure path

↓

Things are rejected correctly

A failed request is not always evidence that the API itself is broken.

Sometimes failure is exactly the correct behavior.

For example:

```
Invalid API key
  ↓
Server rejects request
```

That is usually expected behavior.

6. Successful request

A **successful request** is one where:

```
The client sends acceptable input
      +
Any required authentication is valid
      +
The server successfully processes it
```

Example:

```
GET /products/10
```

with valid authentication.

Possible response:

```
200 OK
```

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

Conceptually:

```
Postman
  |
  | Valid request
  ↓
Server
  |
  | Everything is acceptable
  ↓
200 OK
```

7. Failed request

A **failed request** means the server cannot or should not complete the requested operation.

Examples include:

```
Missing required input
Invalid JSON
Wrong URL
Missing API key
Invalid API key
Resource does not exist
```

Different problems can produce different HTTP status codes.

For example:

```
Missing required data
  ↓
possibly 400 Bad Request

Missing/invalid authentication
  ↓
often 401 Unauthorized

Product not found
  ↓
404 Not Found
```

The exact status code can depend on how a particular API is designed.

8. What is authentication?

Authentication answers:

Who are you?

Imagine entering an office building.

The security desk might ask:

```
Can you prove your identity?
```

In an API, authentication may work similarly.

The server may require something that identifies or verifies the caller.

For today's lesson, that something will be an:

API key

Conceptually:

```
Client
  |
  | "Here is my API key."
  ↓
Server
  |
  | Check key
  ↓
Valid?
```

If valid:

Request may continue

If invalid:

Request may be rejected

9. What is authorization?

Authorization answers a different question:

What are you allowed to do?

So the basic difference is:

```
Authentication
  ↓
Who are you?

Authorization
  ↓
What are you allowed to do?
```

Imagine an employee badge.

Authentication:

"This badge proves I am Alice."

Authorization:

```
"Alice is allowed to enter Room A,  
but not Room B."
```

In API terms:

```
Authentication:  
Is this caller valid?
```

```
Authorization:  
Can this valid caller perform this operation?
```

For Day 125, keep the difference this simple.

10. Authentication vs authorization

A useful mental model is:

```
Step 1  
Who are you?  
  ↓  
Authentication  
  
Step 2  
What can you do?  
  ↓  
Authorization
```

Another simple example:

```
Login successfully  
  ↓  
Authenticated  
  
Try to open admin settings  
  ↓  
Do you have permission?  
  ↓  
Authorization
```

You do not need to learn complex permission systems today.

11. What is an API key?

An **API key** is a value that an API may give to a client so the server can identify or authenticate requests.

Example placeholder:

```
demo_api_key_12345
```

This is only an example.

Never use that as a real security key.

A request might conceptually contain:

```
API key:  
demo_api_key_12345
```

The server checks it:

```
Client request  
  ↓  
API key included  
  ↓  
Server checks key  
  ↓  
Valid or invalid?
```

API-key schemes vary between APIs, so always follow the API's documentation.

12. Authorization header concept

One common place for authentication information is an HTTP header.

For example, conceptually:

```
Authorization: <authentication information>
```

You may eventually see formats such as:

```
Authorization: Bearer ...
```

But you do not need to learn token systems today.

For Day 125, understand only this:

The Authorization header is one common way for a client to send authentication information to a server.

Example using a completely fake placeholder:

```
Authorization: Bearer YOUR_PLACEHOLDER_API_KEY
```

Conceptually:

```
GET /products/10
```

```
Authorization: Bearer YOUR_PLACEHOLDER_API_KEY
```

The server can inspect the header before processing the request.

13. API keys can also use other headers

Not every API puts an API key inside the Authorization header.

Some APIs might use something like:

```
X-API-Key: YOUR_PLACEHOLDER_API_KEY
```

So do not assume every API uses exactly the same format.

The API documentation tells you where the key belongs.

For today's lesson, the important idea is simply:

```
Request
├── method
├── URL
├── headers
│   └── authentication information
└── body if needed
```

14. Why API keys should not be publicly shared

An API key should usually be treated like a secret credential.

Imagine posting your house key publicly.

Anyone finding it could potentially use it.

Similarly, if you publicly expose a real API key, someone else may be able to send requests using your access.

Possible problems include:

```
Unauthorized API usage
Usage limits being consumed
Unexpected costs
Access to protected data
Key being disabled
```

Therefore:

Do not paste real API keys into public GitHub repositories, public screenshots, tutorials, or chat messages.

For examples, use placeholders such as:

```
YOUR_API_KEY
```

or:

```
example_key_123
```

15. Why hard-coding API keys is risky

Suppose you wrote something like:

```
api_key = "real_secret_key_here"
```

directly inside source code.

This is called **hard-coding** the key.

If that file is later uploaded to GitHub or shared with someone, the secret may be exposed.

A safer conceptual approach is:

```
Program
  |
  | asks environment
  ↓
Environment variable
  |
  | provides secret at runtime
  ↓
API request
```

Instead of storing the secret directly in the code.

16. Environment variable idea

An **environment variable** is a value stored outside your main source code that your program can read when it runs.

Imagine:

```
Code:
"Give me the value called API_KEY."
```

```
Environment:  
API_KEY = secret value
```

Then the application uses it.

Conceptually:

```
Environment  
  |  
  | API_KEY  
  ↓  
Application  
  |  
  | uses key  
  ↓  
API request
```

This helps separate:

```
Application code
```

from:

```
Secret configuration
```

For today, you only need the idea.

You do not need to write Python code for environment variables yet.

17. Easy Postman example using a placeholder API key

Imagine this protected endpoint:

```
https://api.example.com/products/10
```

The API requires authentication.

Use this fake placeholder:

```
YOUR_PLACEHOLDER_API_KEY
```

In Postman, your request could conceptually look like:

```
Method:  
GET  
  
URL:  
https://api.example.com/products/10
```

Header:

```
Authorization: Bearer YOUR_PLACEHOLDER_API_KEY
```

Then click:

Send

Conceptually:

```
Postman
|
| GET /products/10
|
| Authorization:
| Bearer YOUR_PLACEHOLDER_API_KEY
↓
Server
|
| Check authentication
|
| Process request
↓
Response
```

18. Successful authenticated example

Assume the placeholder represents a valid key for our imaginary API.

The server might respond:

```
200 OK
```

with:

```
{
  "id": 10,
  "name": "Keyboard",
  "price": 1200
}
```

Flow:

```
Valid request
+
Valid key
↓
Server accepts request
```

↓
200 OK

19. Missing API key example

Now remove the authentication header.

Request:

GET /products/10

but no:

Authorization

header is provided.

Conceptually:

```
Postman
  |
  | GET /products/10
  | no authentication
  ↓
Server
  |
  | authentication required
  ↓
Request rejected
```

A common result may be:

401 Unauthorized

Remember that APIs can differ, so always check the API's documentation.

20. Invalid API key example

Suppose the request contains:

Authorization: Bearer INVALID_EXAMPLE_KEY

The server checks the value.

```
Key provided
  ↓
Server checks key
  ↓
```

```
Key invalid
↓
Request rejected
```

A common response may again be:

```
401 Unauthorized
```

The important testing question is:

Does the API correctly reject invalid authentication?

21. What does API testing mean today?

Today, API testing means deliberately trying a few different inputs and checking the result.

For example:

```
Test 1
Valid request
  ↓
Should succeed

Test 2
Missing required input
  ↓
Should fail appropriately

Test 3
Missing or invalid API key
  ↓
Should reject authentication
```

This simple approach is already much better than testing only one successful request.

22. Problem statement

Imagine a protected product API:

```
POST https://api.example.com/products
```

It requires:

```
A valid API key
```

and product data:

```
{
  "name": "Webcam",
  "price": 2500
}
```

Use this placeholder authentication header:

```
Authorization: Bearer YOUR_PLACEHOLDER_API_KEY
```

Design three manual Postman tests.

Test 1 — Valid request

Send:

```
POST /products
```

with valid placeholder authentication and:

```
{
  "name": "Webcam",
  "price": 2500
}
```

Ask:

```
Should this succeed?
What kind of status/result would you expect?
```

Test 2 — Missing required input

Send the authenticated request but leave out a required value.

For example:

```
{
  "name": "Webcam"
}
```

Assume price is required.

Ask:

```
Should the server create the product?

Or should it reject the request?
```

Test 3 — Missing or invalid API key

Send correct product data but either:

remove the authentication header

or use:

Authorization: Bearer INVALID_EXAMPLE_KEY

Ask:

Should this protected request be accepted?

Your job is to describe the expected behavior.

Do not write application code.

23. Concepts used

Today's exercise combines:

Postman
HTTP request
HTTP response
Headers
JSON body
Status code
API testing
Authentication
Authorization
API key
Environment variable idea

It also builds directly on the previous lessons:

Day 121 → HTTP
Day 122 → HTTP methods
Day 123 → request/response details
Day 124 → Postman
Day 125 → test different API behaviors

24. Thought process

When testing an API, do not randomly change things.

Use one clear question for each test.

For example:

Test question:
Does a correct request work?

Request:
valid input + valid authentication

Then:

Test question:
Does the API reject missing data?

Request:
missing required field

Then:

Test question:
Does the API protect the endpoint?

Request:
missing/invalid authentication

This makes each test easy to understand.

25. Beginner-friendly testing steps

A simple manual API-testing workflow is:

START

Understand what the endpoint expects

Choose HTTP method

Enter URL

Add required headers

Add placeholder API key if authentication is required

Add JSON body if required

Click Send

Inspect status code

Inspect response body

```
Ask:
"Is this the behavior I expected?"

Change one thing for the next test

Send again

Compare the result

END
```

The key phrase is:

```
Change one thing at a time
```

That makes failures much easier to understand.

26. Test 1: valid request

Your first test is normally the **happy path**.

Example:

```
Correct method
Correct URL
Correct authentication
Correct JSON
```

Conceptually:

```
POST /products

Authorization: Bearer YOUR_PLACEHOLDER_API_KEY

{
  "name": "Webcam",
  "price": 2500
}
```

If creation succeeds, a common response would be:

```
201 Created
```

possibly with:

```
{
  "id": 50,
  "name": "Webcam",
```

```
"price": 2500
}
```

Test conclusion:

```
Valid input
  ↓
Successful creation
```

27. Test 2: missing required input

Now remove one required value.

Example:

```
{
  "name": "Webcam"
}
```

Suppose:

```
price
```

is required.

Your test question becomes:

Does the API correctly reject incomplete product data?

A common beginner expectation is a client-error response, such as:

```
400 Bad Request
```

However, the exact status code depends on the API implementation.

The important result is:

```
Product should not be silently accepted
if required input is missing.
```

28. Test 3: missing or invalid API key

Keep the JSON correct:

```
{
  "name": "Webcam",
```

```
"price": 2500
}
```

but remove the authentication information.

Or use:

```
Authorization: Bearer INVALID_EXAMPLE_KEY
```

Your question becomes:

Does the protected API reject callers that are not properly authenticated?

A common result is:

```
401 Unauthorized
```

Conceptually:

```
Correct product data
      +
Invalid authentication
      ↓
Request rejected
```

29. Easy edge case: expired or invalid key

Some authentication credentials may eventually become invalid or expire.

Conceptually:

```
Client sends key
      ↓
Server checks it
      ↓
Key no longer valid
      ↓
Request rejected
```

For today's mental model:

```
Expired key
Invalid key
Missing key
      ↓
Authentication failure
```

You do not need to learn token expiration systems yet.

30. Easy edge case: missing header

Suppose the API expects:

```
Authorization: Bearer YOUR_PLACEHOLDER_API_KEY
```

but the client forgets the entire header.

Then:

```
Postman
  |
  | protected request
  | no Authorization header
  ↓
Server
  |
  | Cannot find required authentication
  ↓
Reject request
```

A common response may be:

```
401 Unauthorized
```

Again, the exact response depends on the API.

31. Expected status/result concept

For your three beginner tests, think approximately like this:

Test	Input	Expected concept
Valid request	Correct data + valid authentication	Success
Missing required input	Incomplete request data	Client/request error
Missing/invalid API key	Authentication problem	Authentication failure

Possible common status codes:

```
Valid creation
  ↓
201 Created

Missing/invalid request data
  ↓
400 Bad Request
```

Missing/invalid authentication

↓

401 Unauthorized

These are useful expectations, but a real API's documentation is the final reference for its exact responses.

32. A useful beginner testing pattern

As your API lessons continue, remember:

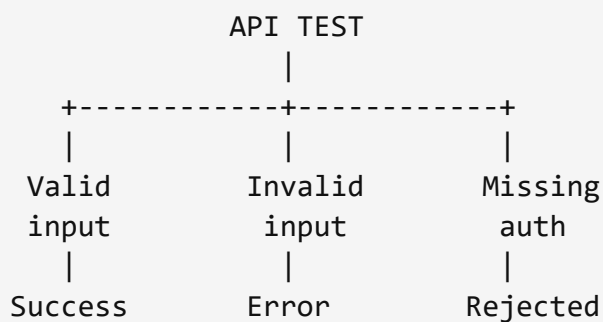
Don't test only:

"Does it work?"

Also test:

"What happens when something is wrong?"

A simple testing triangle is:



This is the beginning of thinking like an API tester and developer.

33. Security habit to remember

Never use a real secret in a learning example.

Use placeholders such as:

YOUR_API_KEY

YOUR_PLACEHOLDER_API_KEY

EXAMPLE_KEY_ONLY

And remember:

Source code

+

Real secret

```
+  
Public repository  
↓  
Dangerous combination
```

Prefer the general idea:

```
Secret  
↓  
Environment/configuration  
↓  
Application
```

instead of:

```
Secret written directly into code
```

34. Hint only

For the three tests, ask one question at a time.

Test 1

Everything is valid.

Think:

```
Create product successfully  
↓  
Which success result fits creation?
```

Test 2

A required field is missing.

Think:

```
Client request is incomplete  
↓  
Should server accept or reject it?
```

Test 3

Authentication information is missing or invalid.

Think:

```
Server cannot authenticate caller  
↓
```

Which authentication-related result makes sense?

Use these familiar codes as clues:

200
201
400
401
404
500

Do not use a real API key.

Day 125 key takeaway

Your API knowledge now connects like this:

Day 121
HTTP request and response
↓
Day 122
GET, POST, PUT, DELETE
↓
Day 123
Parameters, headers, JSON, status codes
↓
Day 124
Send requests using Postman
↓
Day 125
Test successful and failed behavior
+
Understand basic authentication

The main ideas to remember are:

Authentication
↓
Who are you?

Authorization
↓
What are you allowed to do?

And:

API key
↓

Credential used by some APIs

Real API key

↓

Do not publicly share it

Do not casually hard-code it

Environment variable

↓

One way to keep configuration/secrets
outside the main source code

For API testing, develop this habit:

Valid request

↓

Check success

Invalid input

↓

Check correct failure

Missing/invalid authentication

↓

Check that protected access is rejected

That gives you the basic foundation for testing API behavior before moving into more detailed API development and testing techniques.